RFC - Banco de circuito fechado para eventos: credencial, tarifa e cashback

Time	Bruno Budano Mello e Pedro Henrique Mathias Santiago
Data	12/10/2026
Versão	12

Contextualização

Entendendo o problema

O sistema é a infraestrutura de pagamentos de uma feira. O público chega, recebe uma credencial ligada a uma conta e carrega saldo nela. Cada stand também tem conta. A compra acontece pelo app: a pessoa escaneia o produto, confirma, o saldo é debitado e ela sai com o comprovante, sem fila de caixa. O sistema cadastra três tipos de usuário, público, organizadores e empresas expositoras, cria eventos e controla recarga, compra, transferência e estorno entre as contas. Quem chama são o app do público e o painel de organizadores e expositores.

O ganho começa no espaço do stand. Numa feira, parte da área de cada stand vai para o caixa e para a fila que se forma na frente dele, gente parada, esperando a vez de pagar, ocupando o lugar de quem quer ver os produtos. Com o pagamento feito no celular, essa fila deixa de existir. O stand usa a área inteira para expor e atender, e a fila para entrar nele também diminui, porque ninguém fica lá dentro só esperando para pagar.

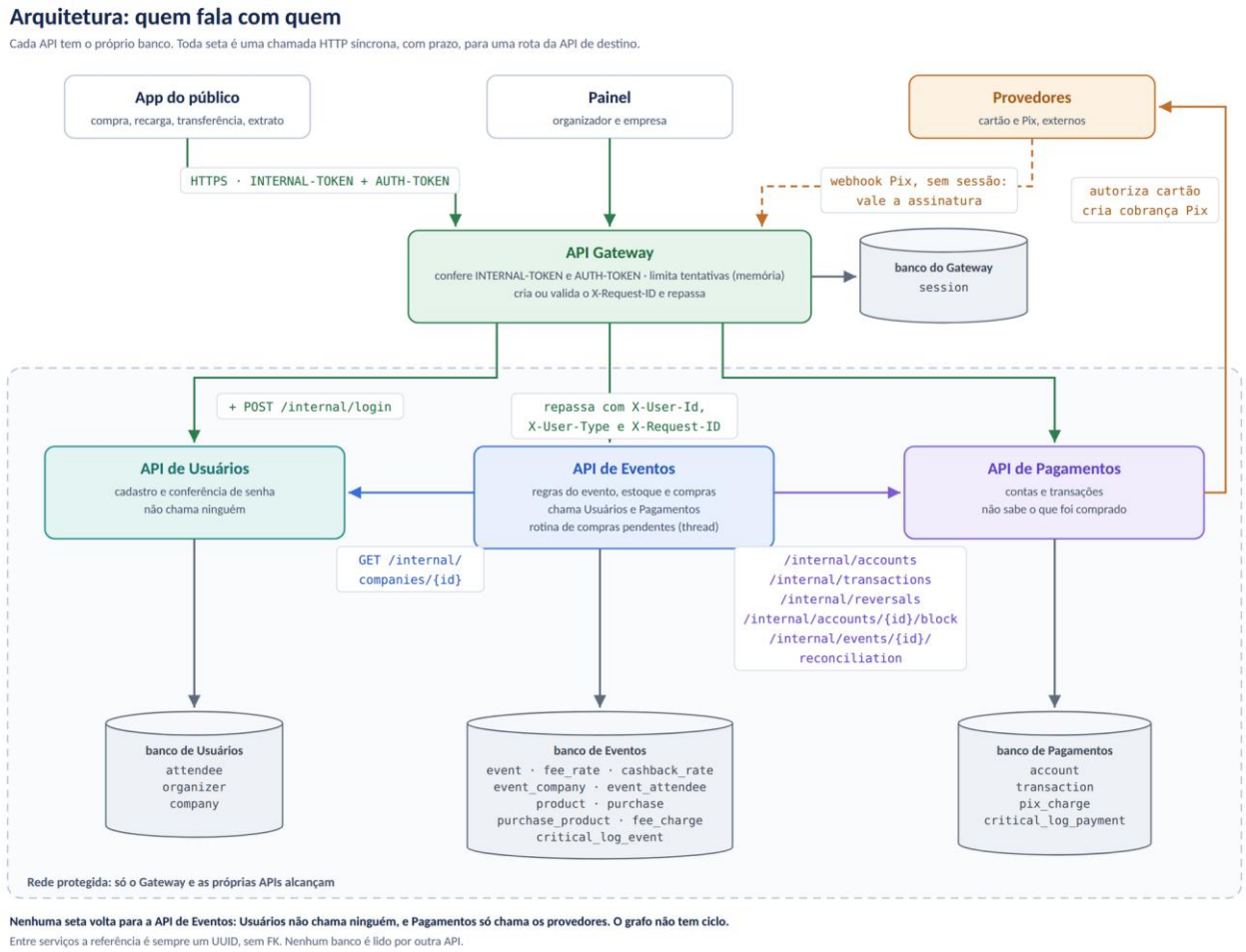
Isso melhora a experiência de todos os lados. O público passa menos tempo parado e mais tempo circulando, e guarda o comprovante de cada compra no próprio celular. Para o expositor, fila menor é venda que não se perde, quem desistiria ao ver a fila do caixa compra na hora, e o atendente que ficaria no caixa volta para o atendimento, o que tende a aumentar o número de vendas. Para o organizador, todas as vendas passam pelo mesmo lugar, a tarifa é cobrada no momento da compra e o fechamento do evento confere, conta por conta, se o dinheiro bate.

Três garantias sustentam tudo. Nenhuma conta de usuário fica negativa, mesmo com duas compras da mesma credencial chegando no mesmo instante. Se essa garantia falhar, o evento entrega produto que ninguém pagou. Nenhuma cobrança acontece duas vezes: a conexão do celular cai, o app reenvia, e a segunda requisição não pode virar uma segunda compra. A soma dos saldos do evento é sempre zero, porque dinheiro não nasce e não some, só muda de conta. Se essa soma deixar de fechar, o extrato do expositor não bate com as vendas dele.

Explicando a solução de forma macro

O sistema é dividido em quatro serviços, cada um com o próprio banco. Nenhum lê o banco do outro e nenhuma tabela tem FK para tabela de outro serviço. Quando uma API precisa de algo que não é dela, pede por HTTP a uma rota interna da dona. A regra que decidiu o corte é uma só: fica junto o que precisa do mesmo COMMIT. Estoque e compra ficam em Eventos. Saldo e transação ficam em Pagamentos. O que só precisa ser consultado de vez em quando, saber se uma empresa existe, mora em outro serviço e é perguntado por HTTP. A rotina que revisa as compras pendentes roda numa thread dentro da API de Eventos, a cada 30 s.

Uma transação tem quatro informações de negócio: a conta de origem, a de destino, o valor e o método (`balance`, `card`, `pix` ou `cash`). Não existe tabela de lançamentos: cada transação já tira de uma conta e põe em outra. A conta guarda o saldo atual para que o débito seja um UPDATE condicional atômico, e o extrato calcula, na própria consulta, o saldo depois de cada movimento. A conciliação confere o saldo guardado contra a soma do histórico.



Quem fala com quem. Toda seta é uma rota com dono, e nenhum banco é compartilhado.

Serviço	O que guarda	Por que é separado
API Gateway	`session`	É a única porta exposta. Confere o `INTERNAL-TOKEN`, troca o `AUTH-TOKEN` pela sessão e cria o `X-Request-ID`. Nenhuma API precisa saber validar sessão. Se a autenticação mudar, muda só ele
API de Usuários	`attendee`, `organizer`, `company`	Concentra o dado sensível: um vazamento de Eventos ou de Pagamentos expõe compras e saldos, mas não senhas, CPFs nem e-mails. Um pedido de exclusão pela LGPD é atendido num lugar só, e a identidade dura mais que um evento
API de Eventos	evento, stands, credenciais, produtos, compras, versões de tarifa e cashback, `fee_charge`	É onde mora toda regra de negócio do evento. Compra e estoque ficam juntos porque reservar o estoque e gravar a compra é uma operação só Roda também a rotina de compras pendentes, numa thread.
API de Pagamentos	account, transaction, pix_charge	Para que o código capaz de criar ou sumir com dinheiro seja o menor possível e mude o mínimo possível. Uma regra nova de cashback ou um tipo novo de produto não tocam nela. É também a única que fala com os provedores de cartão e Pix

Alternativas descartadas

- **Não fazer nada, um caixa e uma maquininha por stand** - é a opção mais barata e a mais comum em feira. Descartada porque não resolve o que o problema pede, continua com filas, o organizador não vê a venda acontecer, cobra a tarifa no fim, pela palavra do expositor, e não tem como conferir o fechamento. Ganharia numa feira pequena, de um dia, com tarifa fixa e ninguém precisando conciliar.
- **Uma API só, com um banco** - descartada porque a regra do evento e o débito de saldo ficariam no mesmo código e no mesmo banco, e qualquer mudança de produto, cashback ou tela passaria perto do dinheiro. Ganharia em simplicidade: uma transação de banco cobriria compra, estoque e débito de uma vez, sem chave de idempotência entre serviços. Seria a escolha certa para um protótipo.

- **Duas APIs, Evento e Conta** - descartada porque a API de Conta precisava saber o que era uma compra, guardava os itens e a regra de tarifa, e perguntava o estoque à outra. A compra ficava espalhada por dois bancos, e existia o caso do estoque reservado num serviço e não devolvido pelo outro. Com a compra inteira em Eventos, esse caso deixou de existir.
- **Usuários dentro da API de Eventos** - descartada porque misturaria dado pessoal com dado do evento no mesmo banco. Ganharia FKs de verdade em `organizer_id` e `company_id` e dispensaria uma chamada. O custo é pequeno, Eventos só chama Usuários para confirmar que uma empresa existe. Seria a escolha certa se o time não conseguisse manter um serviço a mais.
- **Contas e transações em serviços separados** - descartada porque o débito condicional e o registro da transação precisam do mesmo COMMIT: separados, um serviço poderia debitar e o outro não registrar. Ganharia se as contas precisassem escalar separadas das transações, o que não acontece num evento.
- **Tipo da transação gravado em Pagamentos (type)** - descartada no protótipo porque cada movimento já se explica pela chave que o originou (`purchase:`, `refund:`, `cash:`) e pelo kind das contas de origem e destino, e um type obrigaria Eventos e Pagamentos a mudarem juntas a cada tipo novo. Ganharia num extrato que diz o que cada linha foi sem cruzar dados. É o primeiro passo se um extrato detalhado entrar no escopo.
- **Kinds separados para credencial e stand** - descartada porque kind diz o papel contábil da conta (user nunca fica negativa. Fee e external são do sistema), e essa regra vale igual para os dois. Quem diz se o dono é pessoa ou empresa é o `holder_type`. Ganharia se credencial e stand passassem a ter regras de saldo diferentes.
- **Autorizar o cartão antes de gravar qualquer coisa** - descartada porque duas requisições com a mesma chave, chegando juntas, passavam pela consulta prévia e chamavam o provedor duas vezes, a cobrança dupla acontecia fora do nosso banco, onde o UNIQUE não alcança. Ganharia em simplicidade, sem transação pending de cartão e sem um segundo COMMIT.
- **Repetir a resposta original (422 ou 502) no reenvio de uma recarga encerrada** - descartada porque exigiria guardar o motivo da recusa junto da transação. O reenvio devolve a transação como ela está (200), e o app lê o status. Ganharia em seguir à risca o "mesma chave, mesma resposta", que a compra segue: lá o `cancel_code` guarda a recusa e o reenvio a repete.
- **Ordenar o extrato por created_at** - descartada porque o Pix nasce pending no created_at da cobrança e só entra no histórico quando é pago. O saldo das linhas entre os dois momentos mudaria depois de já ter sido mostrado ao cliente.
- **bcrypt ou Argon2, por biblioteca externa** - descartada porque acrescenta uma dependência compilada à imagem, e o scrypt da biblioteca padrão já é lento de propósito e caro em memória, que é o que protege um vazamento contra força bruta. Ganharia em ajuste fino e no Argon2, a recomendação mais nova. SHA-256 com sal foi descartado antes: é rápido demais, e uma placa de vídeo testa bilhões de senhas por segundo.
- **Consultar e-mail e documento antes de gravar o cadastro** - descartada como fonte da verdade porque duas requisições iguais passariam juntas pela consulta. Quem decide a unicidade é o UNIQUE.
- **Devolver o CPF no /me e no cadastro** - descartada por minimização (LGPD), o app não precisa dele, e cada resposta que o carrega é mais um lugar de onde ele pode vazar.
- **Limite de tentativas no banco do Gateway, ou num Redis** - descartado no protótipo porque custa uma escrita por tentativa, ou um serviço a mais, e o Gateway roda numa réplica só. Ganharia com réplicas atrás de um balanceador, em memória, o limite se multiplica pelo número de réplicas e zera quando uma delas reinicia.
- **Rotina de pendentes como processo à parte (events-worker)** - descartada no protótipo porque custa um container e um ponto de entrada a mais, e o compose sobe uma réplica de cada API. Ganharia quando a API de Eventos tiver réplicas, com a thread, cada réplica roda a sua. O trabalho repetido não cobra duas vezes, porque a chave é a mesma, mas multiplica as chamadas a Pagamentos.

Implementação

Comunicação entre os serviços

Toda comunicação é HTTP com JSON, síncrona e com timeout. **Síncrona e não por fila** porque toda chamada interna precisa da resposta para continuar: a compra só fecha se Pagamentos disse que o dinheiro andou. Uma fila acrescentaria um broker para operar e trocaria um erro imediato por uma espera. **Os prazos são escritos e diminuem para dentro:** o Gateway espera até 10 s por uma API, Eventos espera 5 s por Usuários ou Pagamentos, e Pagamentos espera 3 s pelos provedores. Quem está no meio desiste primeiro e devolve um erro com nome (`502 QIT002031`), em vez de deixar quem chamou esperando sem saber por quê.

De fora para dentro, app e painel só enxergam o Gateway. Ele confere o INTERNAL-TOKEN (sem ele, `403 QIT000002`), troca o AUTH-TOKEN pela sessão (sem ela, `401 QIT002002`) e repassa a requisição com X-User-Id, X-User-Type e X-Request-ID (o que veio do app, se passar no filtro de formato, ou um novo). As APIs nunca recebem o token do usuário. O webhook do Pix é a exceção: passa sem sessão e é autenticado pela assinatura do provedor, dentro da API de Pagamentos. O Gateway também limita a taxa de tentativas do login, dos cadastros e do webhook, por IP e por e-mail, com o contador na memória do processo, numa janela deslizante. Com uma réplica, basta. Com várias, cada uma contaria as suas tentativas.

De uma API para outra, pelas rotas /internal/*, que só aceitam chamada de dentro da rede protegida: pelo Gateway elas nem existem (`404 QIT000404`), e quem chega direto na porta de uma API sem o token dela recebe `403 QIT000002`. Eventos e Pagamentos gravam INTERNAL_CALL_REJECTED na própria tabela de log crítico. Usuários, que não tem essa tabela, escreve o mesmo evento como crítico no log da nuvem. Toda chamada interna repassa o X-Request-ID que veio do Gateway, e é ele que liga as linhas de log dos serviços. O X-User-Id não vai nas rotas internas. São poucas e todas numa direção só, o grafo não tem ciclo.

Quem chama	Quem responde	Rota	Para quê	Se não responder
Gateway	Usuários	`POST /internal/login`	Conferir e-mail e senha	`502`. Quem já tem sessão continua usando o sistema
Eventos	Usuários	`GET /internal/companies/{id}`	Confirmar que a empresa existe antes de criar o stand	`502`, e nada é gravado
Eventos	Pagamentos	`POST /internal/accounts`	Abrir as contas do evento, do stand (holder_type company) e da credencial (holder_type attendee)	`502` antes de qualquer escrita local. A chave da conta torna o reenvio seguro
Eventos	Pagamentos	`POST /internal/transactions`	Mover o dinheiro das compras e das recargas em espécie	A compra fica `pending` e é perguntada de novo com a mesma chave
Rotina de pendentes (Eventos)	Pagamentos	`POST /internal/transactions`	Perguntar de novo pelas compras pending, com a mesma chave `purchase:{id}`	Tenta na próxima volta. Depois de 5 minutos, `PURCHASE_OUTCOME_UNKNOWN`
Eventos	Pagamentos	`POST /internal/reversals`	Inverter as transações de um estorno	A compra continua `paid`. O reenvio com a mesma chave completa o estorno
Eventos	Pagamentos	`PUT /internal/accounts/{id}/block`	Bloquear a conta junto com a credencial	`502`, e a credencial não muda sem a conta
Eventos	Pagamentos	`GET /internal/events/{id}/reconciliation`	Conciliar o evento no fechamento	`502`, e nada muda
Pagamentos	Provedores de cartão e Pix	`POST /authorizations` e `POST /charges`	Autorizar o cartão e gerar a cobrança Pix, com a reference da transação	Conexão que não chegou a sair: 502 QIT002031 e a recarga vira canceled. Falha depois do envio: no cartão, a recarga fica pending e vira log crítico. No Pix, pode sobrar uma cobrança órfã, que expira sozinha

Nenhuma transação de banco fica aberta durante uma chamada HTTP. A ordem é sempre a mesma: primeiro o que é remoto, depois o COMMIT local. Na montagem do evento, as contas `fee` e `external` são criadas em Pagamentos e só então o evento e a primeira versão das taxas são gravados e confirmados em Eventos, se o COMMIT local falhar, sobra uma conta sem dono, que o log `ORPHAN_ACCOUNT` registra. No stand e na credencial, o reenvio pede a mesma chave e reaproveita a conta. No evento, não: o id é gerado a cada pedido, então o reenvio abre contas novas, e a que sobrou fica registrada em `ORPHAN_ACCOUNT`. O caminho inverso, segurar a transação aberta esperando a rede, prenderia conexão do pool por tempo indeterminado.

Consistência sem transação distribuída

Cada escrita de dinheiro é um COMMIT local em Pagamentos e cada escrita de compra ou estoque é um COMMIT local em Eventos. O que costura os dois é uma ordem fixa e uma chave:

1. Grava localmente o estado `pending` e faz COMMIT.
2. Chama o outro serviço com uma chave **derivada do que originou a chamada**, nunca gerada na hora.
3. Só então confirma. Se cair entre um passo e outro, fica para trás um registro `pending` que sabe exatamente o que perguntar.

Com a mesma chave, perguntar de novo é seguro: se o trabalho já foi feito, a resposta é a mesma com `200`. Se não, ele é feito agora. Quem garante isso é um `UNIQUE` no banco de quem executa, e não uma consulta prévia, a consulta é só o caminho rápido.

Chave	Quem gera	O que a protege	O que ela impede
`idempotency_key` da compra	O app, uma por compra	`UNIQUE purchase.idempotency_key`	O app reenviar e a compra acontecer duas vezes
`purchase:{id}`	A API de Eventos	`UNIQUE (idempotency_key, leg)` em `transaction`	As três transações de uma compra acontecerem duas vezes
`refund:{id}`	A API de Eventos	`UNIQUE (idempotency_key, leg)` e `uq_transaction_reversal`	O estorno acontecer duas vezes, ou a mesma transação ser invertida duas vezes
`acct:{evento}:fee`, `acct:{evento}:external`, `acct:{evento}:{dono}`	A API de Eventos	UNIQUE account.idempotency_key. Uma fee e uma external por evento e uma conta por dono	A mesma credencial ou o mesmo stand ganhar duas contas. Outra chave para uma conta que já existe recebe 409 QIT002036

Chave	Quem gera	O que a protege	O que ela impede
Chave da recarga e da transferência (UUID)	O app	UNIQUE (idempotency_key, leg) em transaction. O schema das rotas públicas só aceita UUID	Recarga ou transferência repetida por clique duplo ou conexão caída, e o app ocupar uma chave purchase: ou refund: de Eventos
`cash:{UUID v5 do evento e da chave do painel}`	A API de Eventos	UNIQUE (idempotency_key, leg) em transaction	A recarga em espécie lançada duas vezes. A chave do painel só vale dentro do evento

As chaves com prefixo (`purchase:`, `refund:`, `cash:`, `acct:`) só valem nas rotas internas e cabem nos 80 caracteres da coluna. Nas rotas públicas de Pagamentos (`/recharges` e `/transfers`), a chave é um UUID gerado pelo app, se fosse texto livre, a empresa poderia transferir 1 centavo com a chave `refund:{id}` de uma venda própria e travar o estorno dela para sempre. O espaço de chaves é um só na API: a mesma chave usada por duas pessoas dá 409 QIT002025 na segunda. As chaves da compra e da recarga em espécie podem ser texto livre porque nunca chegam a Pagamentos como vieram: a da compra fica numa coluna de Eventos, e a da recarga em espécie vira `cash:{UUID v5}`.

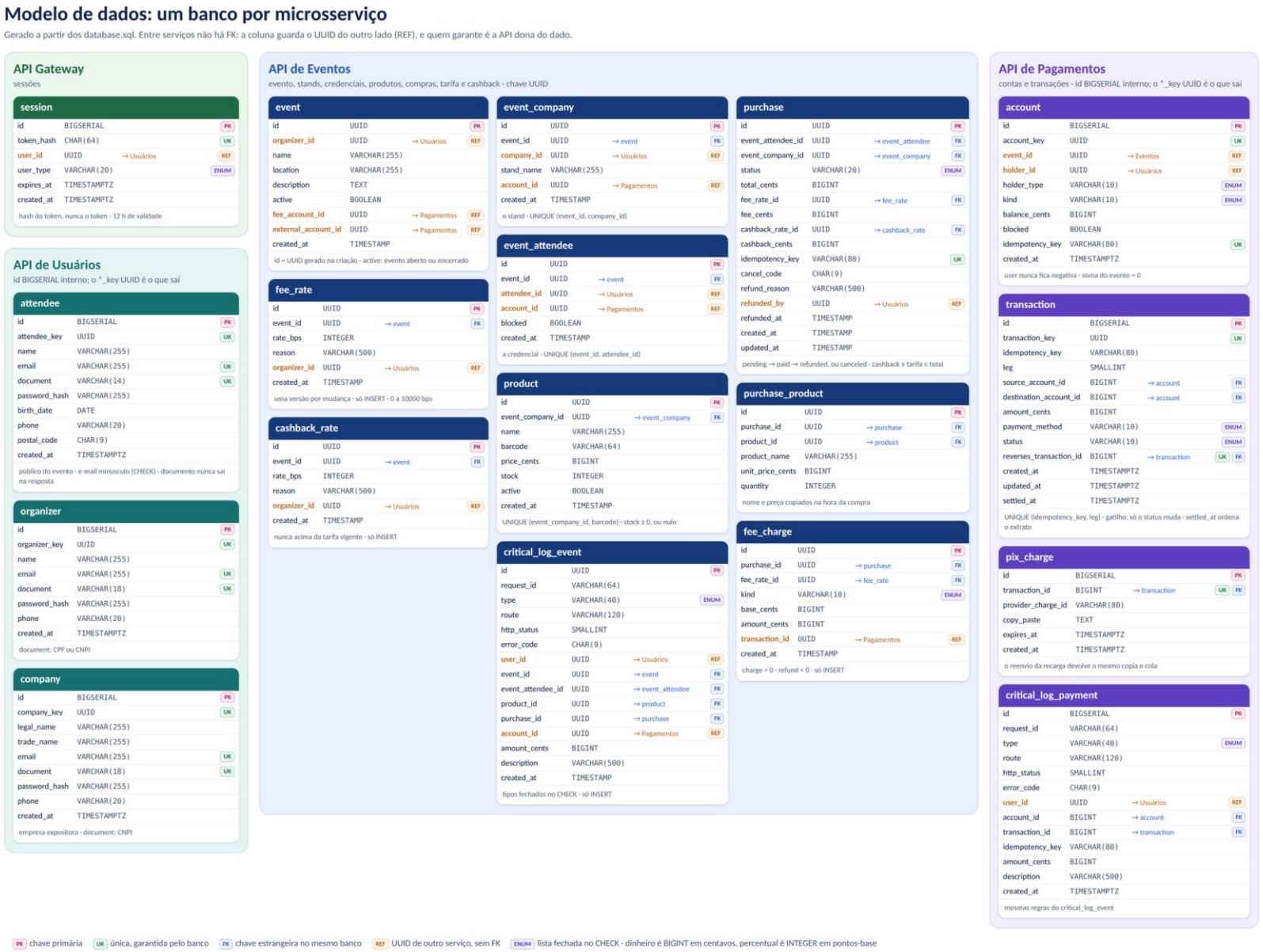
Convenções e decisões de modelagem

- **Dinheiro é BIGINT em centavos** e percentual é INTEGER em pontos-base (500 = 5%). Ponto flutuante não representa 0,10 exatamente, e o resíduo se acumula até o extrato não bater. Tarifa e cashback saem de divisão inteira, sempre para baixo (`total × bps // 10000`), o que garante cashback ≤ tarifa ≤ total: 5% de R$ 1,99 dá 9 centavos. Cada perna de transação tem teto de 10000000 centavos (R$ 100 mil), e o preço do produto também. O estoque vai até 1.000.000 unidades. Eventos recusa antes de chamar Pagamentos a compra cujo total passe do teto (`422 QIT002039`). Valor ≤ 0 é `422 QIT002028` em toda rota de dinheiro.
- **Nenhuma chave sequencial sai de um serviço.** Em Eventos, a chave primária é um UUID gerado pela aplicação, e é ele que sai na resposta. Em Usuários e em Pagamentos, a PK é um id BIGSERIAL interno, e uma coluna `<tabela>_key` UUID é o que sai da API e o que os outros serviços guardam. O id interno nunca aparece numa resposta. Os dois desenhos cumprem a mesma regra, não deixar varrer nem vazar volume, e a referência que atravessa serviços (`account.holder_id`, `event.fee_account_id`) é sempre um UUID. Em produção o UUID seria v7, ordenável por tempo, para não fragmentar o índice.
- **transaction tem FK para account** nas duas pontas. As duas tabelas estão no mesmo banco, então a FK custa nada e impede transação apontando para conta que não existe. A regra "sem FK" vale entre serviços, não dentro de um.
- **A tarifa incide sobre a transferência que representa uma venda**, que é a compra: o stand recebe o valor cheio e devolve a tarifa ao evento, em transações próprias e visíveis no extrato dele. Transferência entre duas credenciais não cobra tarifa porque não houve venda. Se o evento quiser cobrar, o lugar de decidir é uma versão nova de `fee_rate`, e nenhuma outra coisa muda.
- **Recurso que não é de quem chama responde 404, nunca 403.** Com `403`, varrer chaves diz quais existem, e o mapa das credenciais da feira sai de uma varredura. O `403` fica só para "este tipo de usuário não faz isso", que não confirma a existência de nada.
- **fee_rate e cashback_rate não aceitam UPDATE nem DELETE**: um gatilho recusa os dois. Mudar uma taxa é inserir uma versão nova (`PUT /events/{id}/fee-rate`) com a justificativa, quem decidiu e quando. Cada compra guarda a versão que usou, e o passado nunca é recalculado. `fee_charge` é a auditoria do que o sistema recebeu, com uma linha negativa quando a compra é estornada.
- **Transação não tem tipo.** Pagamentos guarda origem, destino, valor, método, status e, no estorno, a transação que ela inverte. O que o movimento foi se lê pela chave que o originou (`purchase:`, `refund:`, `cash:`) e pelo kind das contas. A alternativa de gravar um type está nas alternativas descartadas. Numa transação só o status muda, e um gatilho trava as transições: pending → settled | canceled e settled → reversed. O `settled_at`, preenchido por gatilho no momento em que o dinheiro anda, ordena o extrato.
- **Contas do sistema não têm dono.** `holder_id` e `holder_type` só existem em contas user: `attendee` na credencial e `company` no stand. fee e external são do evento, uma de cada, e nenhuma rota pública as alcança. Elas só aparecem na conciliação. Conta bloqueada não paga nem recebe transferência, mas recarga autorizada, Pix pago e estorno movem o dinheiro mesmo assim: a pessoa já pagou, ou o dinheiro está voltando.
- **Rotas seguem REST: substantivo em inglês, no plural, e a intenção no método.** O extrato é `GET /accounts/{id}/transactions`, porque o recurso é a transação. Statement seria um relatório. Mudar uma taxa é `PUT /events/{id}/fee-rate`: cada chamada grava uma versão nova, e a anterior fica no histórico. Como repetir a chamada grava outra versão, `POST /events/{id}/fee-rates` seria o nome mais fiel ao REST, e esse é um trade-off conhecido da rota. Editar um produto é `PUT /products/{id}`, que muda só os campos enviados. Ação sem substantivo confortável usa o mais próximo: `/purchases/{id}/refund` é o estorno, `/events/{id}/reconciliation` é o fechamento, e `PUT /event-attendees/{id}/block` grava o estado de bloqueio, que repetir não altera. Paginação vai na query string, com `limit` e `page` a partir de 1.
- **O evento nasce aberto.** As rotas de stand, credencial, compra e recarga em espécie recusam evento encerrado (`409 QIT002011`). A rota que encerra o evento fica fora do escopo deste protótipo, porque depende da decisão sobre o saque da sobra de saldo (veja Perguntas em aberto).

- **A sessão fica no banco do Gateway, e nenhuma API guarda estado de negócio na memória.** Há duas exceções, de propósito: o contador de tentativas do Gateway e a rotina de pendentes de Eventos vivem no processo. Antes de pôr réplicas atrás de um load balancer, que pergunta ao `/health_check` quem está de pé, os dois precisam sair dele (veja as alternativas descartadas).

Os valores fechados são CHECK com a lista, e não ENUM do Postgres, que não deixa remover valor: `payment_method` (balance, card, pix, cash), `transaction.status` (pending, settled, reversed, canceled), `account.kind` (user, fee, external), `account.holder_type` (attendee, company), `purchase.status` (pending, paid, canceled, refunded) e `fee_charge.kind` (charge, refund).

Banco de Dados (Somente diagrama)



Um banco por microsserviço, desenhado a partir dos `database.sql`. Nenhuma FK atravessa a fronteira entre serviços. Dentro de cada banco, as FKs existem.

Rotas

Todas as rotas públicas passam pelo Gateway. As /internal/* não existem por ele. Não pedem `AUTH-TOKEN`: o login, os três cadastros e o webhook do Pix, que se autentica pela assinatura. As rotas `/` e `/health_check` existem em todas as APIs e não pedem token.

API Gateway

Método	Caminho	O que faz	Entrada	Saídas
POST	/auth/login	Recebe email, password e user_type, pede à API de Usuários que confira, gera um token aleatório, grava a sessão com o hash dele e devolve o token. Limita tentativas por e-mail e por IP	email, password, user_type	200 com token, user_id, user_type e expires_at. QIT000001, QIT002001, QIT002003 tentativas demais, QIT002031

API de Usuários

Método	Caminho	O que faz	Entrada	Saídas
POST	/attendees	Cadastra uma pessoa do público	name, email (ASCII), document (CPF com máscara), password, birth_date, phone, postal_code	201 com o cadastro, sem senha e sem documento. QIT000001, QIT002006, QIT002007, QIT002005, QIT002008 (abaixo de 18 anos ou data inválida)
POST	/organizers	Cadastra um organizador	name, email (ASCII), document (CPF ou CNPJ com máscara), password, phone	201 com o cadastro, sem senha e sem documento. QIT000001, QIT002006, QIT002007, QIT002005
POST	/companies	Cadastra uma empresa expositora	legal_name, trade_name, email (ASCII), document (CNPJ com máscara), password, phone	201 com o cadastro, sem senha e sem documento. QIT000001, QIT002006, QIT002007, QIT002005
GET	/me	Devolve quem é o dono do token, lido na tabela do tipo dele	-	200 com o cadastro de quem chama, sem senha e sem documento. QIT002002 sem X-User-Id, usuário inexistente ou X-User-Type diferente da tabela
POST	/internal/login	Confere e-mail e senha na tabela do tipo informado · Gateway	email, password, user_type	200 com id e user_type, e nada mais do cadastro. QIT002001, QIT000002
GET	/internal/companies/{id}	Diz se a empresa existe · API de Eventos	-	200 com id, legal_name e trade_name. QIT002009 empresa inexistente, id de outro tipo ou malformado. QIT000002

API de Eventos

Método	Caminho	O que faz	Entrada	Saídas
POST	/events	Cria o evento com a tarifa e o cashback iniciais. A API de Pagamentos abre as contas fee e external antes do COMMIT · organizador	name, location, description, fee_rate_bps, cashback_rate_bps	201. QIT000001, QIT002004, QIT002032, QIT002031
GET	/events	Lista eventos, paginado	limit, page	200 com data e is_last_page
GET	/events/{id}	Devolve o evento com a tarifa e o cashback vigentes	-	200. QIT002010
PUT	/events/{id}/fee-rate	Grava uma versão nova da tarifa, com justificativa. Nunca edita a anterior · organizador do evento	rate_bps, reason	201 com a versão. QIT000001, QIT002010, QIT002032
PUT	/events/{id}/cashback-rate	Grava uma versão nova do cashback, com justificativa. Nunca edita a anterior · organizador do evento	rate_bps, reason	201 com a versão. QIT000001, QIT002010, QIT002032
GET	/events/{id}/cashback-rates	Histórico do cashback: cada versão com taxa, justificativa, autor e data · quem participa do evento	-	200. QIT002010
GET	/events/{id}/fee-charges	Auditoria da tarifa recebida: cada cobrança e devolução, com a versão aplicada · organizador do evento	limit, page	200. QIT002010
POST	/events/{id}/companies	Vincula uma empresa ao evento. Confere na API de Usuários que ela existe e pede a conta do stand à API de Pagamentos · organizador do evento	company_id, stand_name	201. QIT000001, QIT002004, QIT002009, QIT002010, QIT002011, QIT002012, QIT002031
POST	/events/{id}/attendees	A pessoa pega a credencial. A conta dela nasce na API de Pagamentos · público	nenhum: a pessoa vem da sessão	201. QIT002004, QIT002010, QIT002011, QIT002013, QIT002031

Método	Caminho	O que faz	Entrada	Saídas
PUT	/event-attendees/{id}/block	Bloqueia ou libera a credencial e a conta dela, juntas · **idempotente**	blocked	200. QIT002014, QIT002031
POST	/products	Cadastra um produto no stand · empresa do stand	event_company_id, name, price_cents (até 10000000), barcode, stock	201. QIT000001, QIT002004, QIT002010, QIT002018, QIT002028
GET	/products	Vitrine e scanner, paginado	event_id, event_company_id, barcode, name, limit, page	200. QIT000001
GET	/products/{id}	Devolve um produto	-	200. QIT002016
PUT	/products/{id}	Muda só os campos enviados: preço, estoque, nome ou disponibilidade. Não reescreve compra feita. O estoque só muda se ainda for o valor lido em expected_stock · empresa do stand	price_cents, stock, expected_stock, active, name	200. QIT000001, QIT002016, QIT002038 estoque mudou desde a leitura, QIT002028
POST	/purchases	Compra pelo app: reserva o estoque, pede as transações à API de Pagamentos e devolve o comprovante · público · **idempotente**	event_attendee_id, event_company_id, items[{product_id, quantity}], idempotency_key	201 com o comprovante. 200 no replay. QIT000001, QIT002004, QIT002011, QIT002014, QIT002015, QIT002016, QIT002017, QIT002019, QIT002021, QIT002025, QIT002031, QIT002039 total acima do teto
GET	/purchases/{id}	O comprovante: itens, total, tarifa, cashback e status	-	200. QIT002023
POST	/purchases/{id}/refund	Estorno: pede à API de Pagamentos que inverta as transações da compra · quem comprou, a empresa do stand ou o organizador	reason	201 com o comprovante do estorno (motivo, autor e data). QIT000001, QIT002023, QIT002026, QIT002021, QIT002031
POST	/events/{id}/cash-recharges	Recarga em dinheiro: da conta external do evento para a conta da credencial, método cash, sob a chave cash: · organizador do evento · **idempotente**	event_attendee_id, amount_cents (até 10000000), idempotency_key	201 com o comprovante. 200 no replay. QIT000001, QIT002004, QIT002010, QIT002011, QIT002014, QIT002015, QIT002025, QIT002028, QIT002031
GET	/events/{id}/reconciliation	Fechamento: pede a conciliação do evento à API de Pagamentos · organizador do evento	-	200 com sum_cents, accounts, mismatches e o resumo das compras. QIT002010, QIT002031

API de Pagamentos

Método	Caminho	O que faz	Entrada	Saídas
POST	/recharges	Recarga por cartão ou Pix na conta de quem chama. Cartão liquida na hora. Pix fica pending até o webhook · dono da conta · **idempotente**: o reenvio devolve a recarga como ela está	account_id, payment_method (card/pix), amount_cents (até 10000000), card_token (só cartão), idempotency_key (UUID)	201 settled, ou pending com pix_copy_paste e expires_at. 200 no replay, com o status atual. QIT000001, QIT002020, QIT002022, QIT002025, QIT002037 evento sem external, QIT002028, QIT002033, QIT002031
PUT	/webhooks/pix/{id}	Chamada pelo provedor, sem AUTH-TOKEN. Pago: credita e liquida. Expirado: cancela · provedor Pix · **idempotente**	status (paid/expired), amount_cents (qualquer valor que caiba no BIGINT), reference (o id da transação), assinatura no cabeçalho	204. QIT000001. QIT002034 assinatura inválida ou aviso de outra cobrança. QIT002023 (inclui id de recarga de cartão). QIT002024. QIT002035 valor diferente do cobrado, inclusive acima do teto
POST	/transfers	Move saldo de uma credencial para outra, no mesmo evento. Sem tarifa e sem cashback · dono da conta de origem · **idempotente**	source_account_id, destination_account_id, amount_cents (até 10000000), idempotency_key (UUID)	201 com o comprovante. 200 no replay. QIT000001, QIT002004 a origem não é credencial, QIT002020, QIT002022 origem bloqueada, QIT002021, QIT002027 destino não aceita (stand, fee, external ou credencial bloqueada), QIT002029, QIT002030
GET	/accounts	As contas de quem chama	-	200
GET	/accounts/{id}	Saldo da conta	-	200. QIT002020
GET	/accounts/{id}/transactions	Extrato paginado, feito só das transações da conta, na ordem em que o dinheiro andou, com o saldo depois de cada uma	limit (1 a 100), page (a partir de 1), date_from, date_to (sem fuso, em America/Sao_Paulo, e date_to só com data inclui o dia inteiro)	200. QIT000001 (inclui date_from depois de date_to), QIT002020
GET	/transactions/{id}	Uma transação: origem, destino, valor, método e status	-	200. QIT002023
POST	/internal/accounts	Cria uma conta user, fee ou external com saldo zero. Uma fee e uma external por evento, e uma	event_id, kind, idempotency_key. holder_id	201 com a conta. 200 com a mesma conta. QIT000001, QIT000002, QIT002025, QIT002036

Método	Caminho	O que faz	Entrada	Saídas
		conta por dono em cada evento · API de Eventos · **idempotente**	e holder_type (attendee ou company) só em user	o evento já tem essa conta, com outra chave
POST	/internal/transactions	Executa de 1 a 20 transações juntas, todas ou nenhuma, na ordem enviada · API de Eventos · **idempotente**	idempotency_key, payment_method, transfers[{source_account_id, destination_account_id, amount_cents (até 10000000)}]	201 com as transações. 200 no replay. QIT000001, QIT002020, QIT002022, QIT002021, QIT002025, QIT002028, QIT002029, QIT002030, QIT000002
POST	/internal/reversals	Inverte todas as transações de uma chave, uma vez só, começando pelas que saem de contas do sistema. Ignora o bloqueio das contas · API de Eventos · **idempotente**	original_idempotency_key, idempotency_key	201. 200 no replay. QIT002023, QIT002026, QIT002021, QIT000002
PUT	/internal/accounts/{id}/block	Bloqueia ou libera a conta. Bloqueada, não paga nem recebe transferência. Recarga autorizada, Pix pago e estorno movem o dinheiro mesmo assim · API de Eventos · **idempotente**	blocked	200. QIT002020, QIT000002
GET	/internal/events/{id}/reconciliation	Soma os saldos do evento e compara cada saldo com o histórico · API de Eventos	-	200 com sum_cents, accounts e mismatches. QIT000002

Erros que toda rota pode devolver, além dos listados: `400 QIT000001` (corpo ou query fora do schema), `400 QIT000010` (em Eventos, parâmetro de caminho malformado. Em Usuários e Pagamentos, um id malformado responde 404, igual ao inexistente, e as duas formas cumprem a R8), `403 QIT000002` (INTERNAL-TOKEN ausente ou errado), `404 QIT000404` (rota que não existe), `405 QIT000405` (método que a rota não aceita) e `500 QIT000500` (o erro que ninguém nomeou, sempre com log). O corpo de erro tem sempre os mesmos quatro campos: title, description, translation e code.

Logs críticos

Duas tabelas, uma em cada banco que escreve dinheiro ou regra, com a mesma forma: `request_id`, `type`, rota, status, código de erro, os ids que explicam o caso e a descrição. Só aceitam INSERT, e o `type` é uma lista fechada no `CHECK`, log sem tipo previsto é bug, não linha nova. Cada tipo exige os campos que o explicam.

Em Eventos e em Pagamentos, o log é gravado numa sessão de banco própria, e por isso fica mesmo quando a requisição que o causou é desfeita. Usuários não tem tabela: escreve o INTERNAL_CALL_REJECTED como crítico no log da nuvem.

Tabela	Tipo	Quando	Campo que o explica
ambas	INTERNAL_CALL_REJECTED	Uma rota /internal/* foi chamada de fora da rede interna	http_status = 403
ambas	UNEXPECTED_ERROR	Erro 500 numa rota que escreve	http_status = 500
evento	ORPHAN_ACCOUNT	A conta foi criada, mas o COMMIT do evento, do stand ou da credencial falhou depois	account_id
evento	CREDENTIAL_BLOCK_CHANGED	Alguém bloqueou ou liberou uma credencial, o que tira ou devolve o acesso de uma pessoa ao próprio saldo	event_attendee_id e quem fez
evento	STOCK_CHECK_TRIGGERED	O CHECK de estoque barrou uma escrita: o UPDATE condicional deixou passar algo, e isso é bug	product_id
evento	PURCHASE_OUTCOME_UNKNOWN	Pagamentos não respondeu e a compra continua pending depois de 5 minutos de novas tentativas. Quem tenta é a rotina de pendentes da API de Eventos, a cada 30 s, sempre com a mesma chave	purchase_id
evento	RECONCILIATION_MISMATCH	A conciliação achou soma diferente de zero ou saldo diferente do histórico	event_id e a diferença
pagamento	BALANCE_CHECK_TRIGGERED	O CHECK de saldo barrou uma escrita: o débito condicional deixou passar algo, e isso é bug	account_id
pagamento	CARD_AUTHORIZED_NOT_CREDITED	O provedor aprovou, mas a gravação falhou: a pessoa pagou e não recebeu saldo	idempotency_key, quem e quanto
pagamento	CARD_OUTCOME_UNKNOWN	Falha depois de o pedido sair (prazo de resposta, conexão derrubada, resposta cortada): não se sabe se a	idempotency_key, quem e quanto

Tabela	Tipo	Quando	Campo que o explica
		pessoa foi cobrada, e a recarga fica pending. Ação: consultar o provedor pela reference e concluir à mão, num único COMMIT, settled com o saldo, se a autorização foi aprovada. Canceled, se não foi	
pagamento	`PIX_INVALID_SIGNATURE`	Aviso com assinatura válida que aponta para outra cobrança: indício de fraude. Assinatura ausente ou errada vai só para o log comum, e o Gateway limita a taxa da rota	`http_status = 401`
pagamento	`PIX_AMOUNT_MISMATCH`	Valor pago diferente do cobrado, inclusive acima do teto de recarga	`transaction_id` e o valor pago

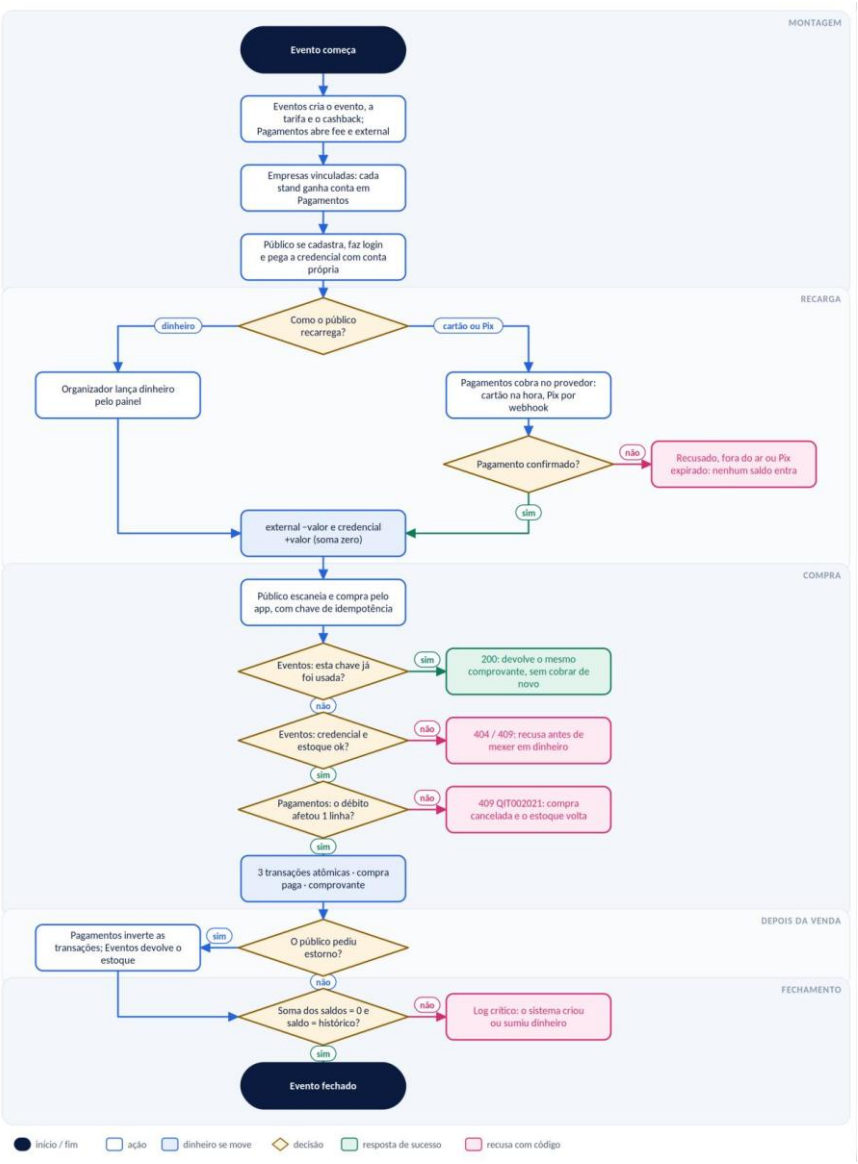
Fora a tabela, toda API escreve `ENTROU` e `SAIU` de cada requisição, com método, caminho, status e tempo em ms, e o X-Request-ID em cada linha. É por ele que "a compra falhou ontem às 15h" vira uma busca: o container daquela hora já não existe, mas o log dele foi para fora.

Alarmes: de sinal a ação

Log crítico que ninguém lê não protege nada. Cada sinal vira um alarme com regra, prioridade e quem age, e todo alarme pede uma ação: P1 acorda o plantão, P4 espera o próximo dia útil. CPU, memória e tempo de resposta são medidos pela infraestrutura, fora do repositório.

Sinal	Regra	Prioridade	Quem age, e como
`BALANCE_CHECK_TRIGGERED` · `STOCK_CHECK_TRIGGERED`	qualquer ocorrência	P1	O débito ou a baixa condicional deixou passar o que o CHECK barrou: é bug na escrita mais protegida do sistema. Congela deploy e segue o request_id.
`RECONCILIATION_MISMATCH`	qualquer ocorrência	P1	Dinheiro criado ou sumido no evento. As vendas daquele evento param até a diferença ter explicação.
Taxa de 5xx em `POST /purchases`, `/recharges` ou `/transfers`	acima de 1% por 5 minutos seguidos	P1	É o 500 que ninguém nomeou, em escala. O plantão lê os `UNEXPECTED_ERROR` do período e segue o request_id.
CPU ou memória do container	CPU acima de 80% por 10 minutos, ou memória que só sobe	P1	Não espera o cliente reclamar: sobe réplica, ou derruba a que vaza memória.
`CARD_AUTHORIZED_NOT_CREDITED` · `CARD_OUTCOME_UNKNOWN`	qualquer ocorrência	P2, no mesmo dia	A pessoa pode ter pago sem receber saldo. Consulta o provedor pela reference, que é a chave da recarga, e conclui à mão.
`PURCHASE_OUTCOME_UNKNOWN`	qualquer ocorrência	P2	Pagamentos ficou 5 minutos sem responder por aquela compra: confere a chave `purchase:{id}` em Pagamentos e se a API de Eventos está de pé.
`PIX_INVALID_SIGNATURE` · `PIX_AMOUNT_MISMATCH`	qualquer ocorrência	P2	Indício de fraude ou de provedor mal configurado: a recarga não anda até alguém conferir com o provedor.
`INTERNAL_CALL_REJECTED`	mais de 10 em 5 minutos	P3	Alguém dentro da rede tentando adivinhar token, ou um deploy com o token errado.
`ORPHAN_ACCOUNT`	qualquer ocorrência	P4, próximo dia útil	Conta sem dono e sem saldo, sem risco: confirma que o reenvio a reaproveitou.
`CREDENTIAL_BLOCK_CHANGED`	não alarma	-	É trilha de auditoria: responde quem bloqueou, quem liberou e quando.

Fluxos



Visão geral: retângulo é ação, losango é decisão, verde é sucesso e rosa é recusa com código.

Nos diagramas de sequência, verde é o Gateway, verde-azulado é a API de Usuários, azul é a de Eventos e roxo é a de Pagamentos. As faixas numeradas são as etapas do fluxo, na ordem em que acontecem.

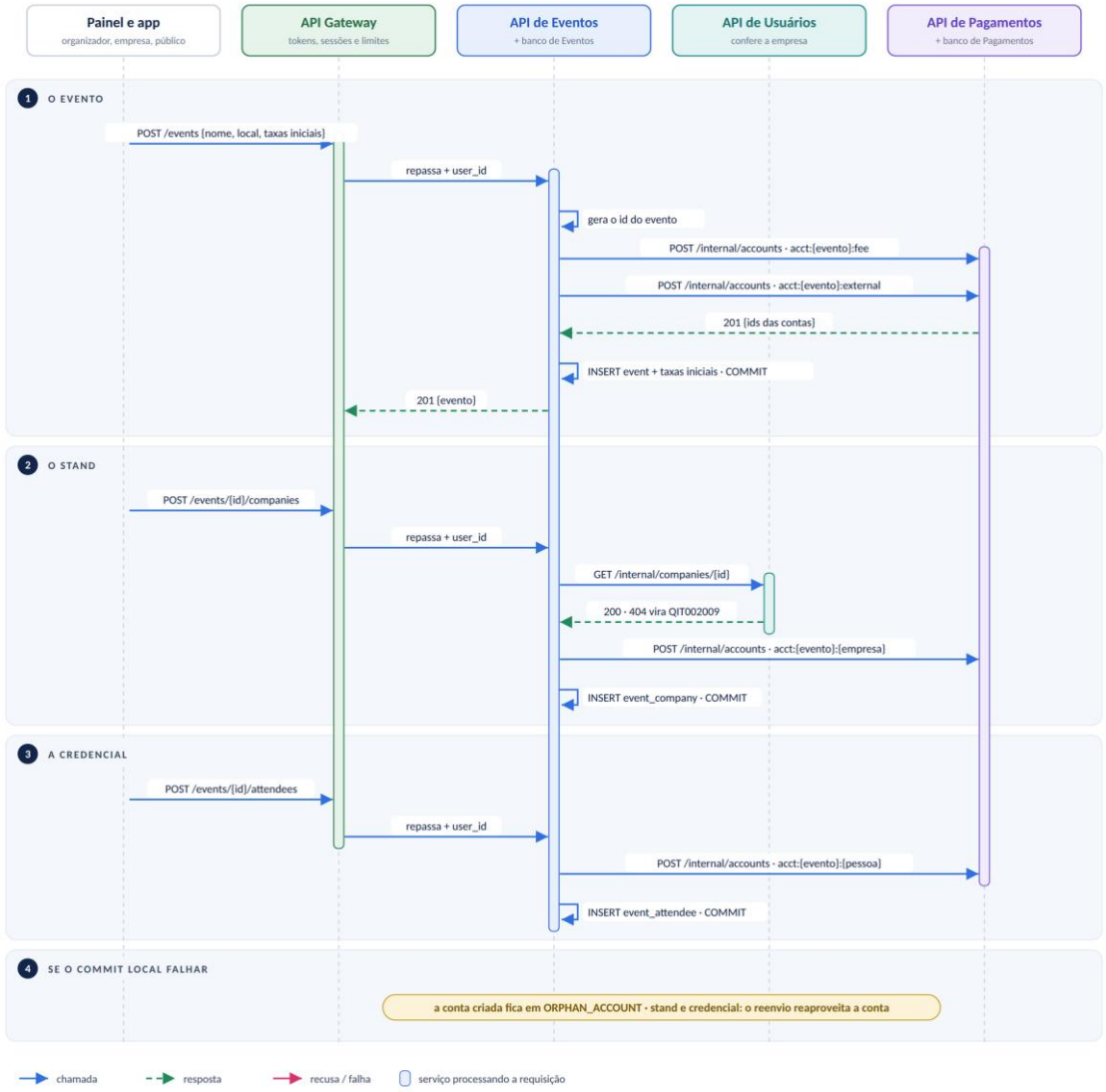
Cadastro e login



Usuários guarda o dado sensível. Os outros serviços só conhecem o UUID.

1. O cadastro passa pelo Gateway sem sessão. Usuários confere o formato, o e-mail só aceita ASCII (letras, dígitos e . _ % + -), para que a forma minúscula tenha o mesmo tamanho e caiba na coluna, os dígitos do CPF ou do CNPJ, se a data de nascimento existe e a idade mínima de 18 anos, contada na data de hoje em America/Sao_Paulo (`422 QIT002008`).
2. Quem decide se o e-mail e o documento são únicos naquela tabela é o UNIQUE do banco (`409 QIT002006`, `QIT002007`), e não uma consulta prévia: o nome da constraint violada diz qual dos dois foi. O e-mail é guardado em minúsculas (um CHECK garante), e o documento com a máscara, que nunca sai nas respostas.
3. A senha é gravada só em hash e conferida em tempo constante. Senha errada e e-mail inexistente recebem a mesma resposta, `401 QIT002001`, e levam o mesmo tempo: sem o e-mail, a API calcula o hash mesmo assim.
4. Cada tentativa custa ~16 MiB e algumas dezenas de ms. É o preço de proteger as senhas num vazamento, e por isso Usuários roda no máximo 4 hashes ao mesmo tempo e o Gateway limita tentativas por IP e por e-mail, com o contador na memória do Gateway. O cadastro ainda revela se um e-mail já tem conta naquele tipo (`409 QIT002007`): preço aceito, porque escondê-lo exigiria confirmar o e-mail, fora do escopo.
5. No login, o Gateway recebe de Usuários só o id e o user_type, gera um token aleatório e grava a sessão com o hash dele.

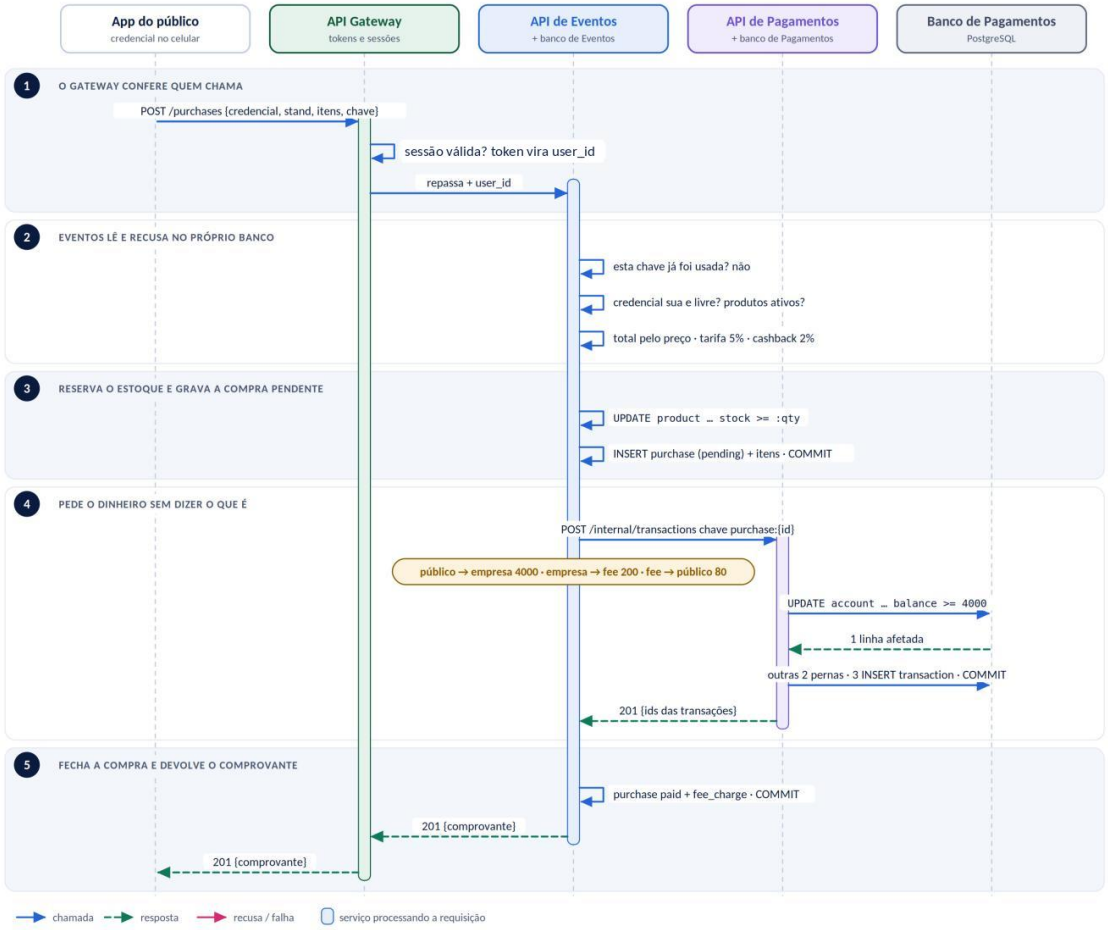
Montagem do evento



Primeiro o que é remoto, depois o COMMIT local: a conta nasce antes do registro que a usa.

1. O organizador cria o evento. Eventos gera o id do evento, pede a Pagamentos as contas fee e external com as chaves `acct:{evento}:fee` e `acct:{evento}:external` e só então grava o evento e a primeira versão das taxas, num COMMIT só. É uma de cada por evento: outra chave para a mesma conta recebe `409 QIT002036`.
2. Para o stand, Eventos confere em Usuários que a empresa existe (`404 QIT002009`) e pede a conta com a empresa como holder_id e holder_type company. A credencial é pedida pela própria pessoa, e a conta dela nasce com holder_type attendee.
3. Se o COMMIT local falhar depois que a conta nasceu, sobra uma conta sem dono, registrada em ORPHAN_ACCOUNT. No stand e na credencial, o reenvio pede a mesma chave e reaproveita a conta. No evento, não: o id é gerado a cada pedido, então o reenvio abre contas novas, e a que sobrou fica registrada em `ORPHAN_ACCOUNT`.
4. O evento nasce aberto. As rotas de stand, credencial, compra e recarga em espécie recusam evento encerrado com `409 QIT002011`. A rota que encerra o evento fica fora do escopo deste protótipo (veja Perguntas em aberto).

Compra no stand

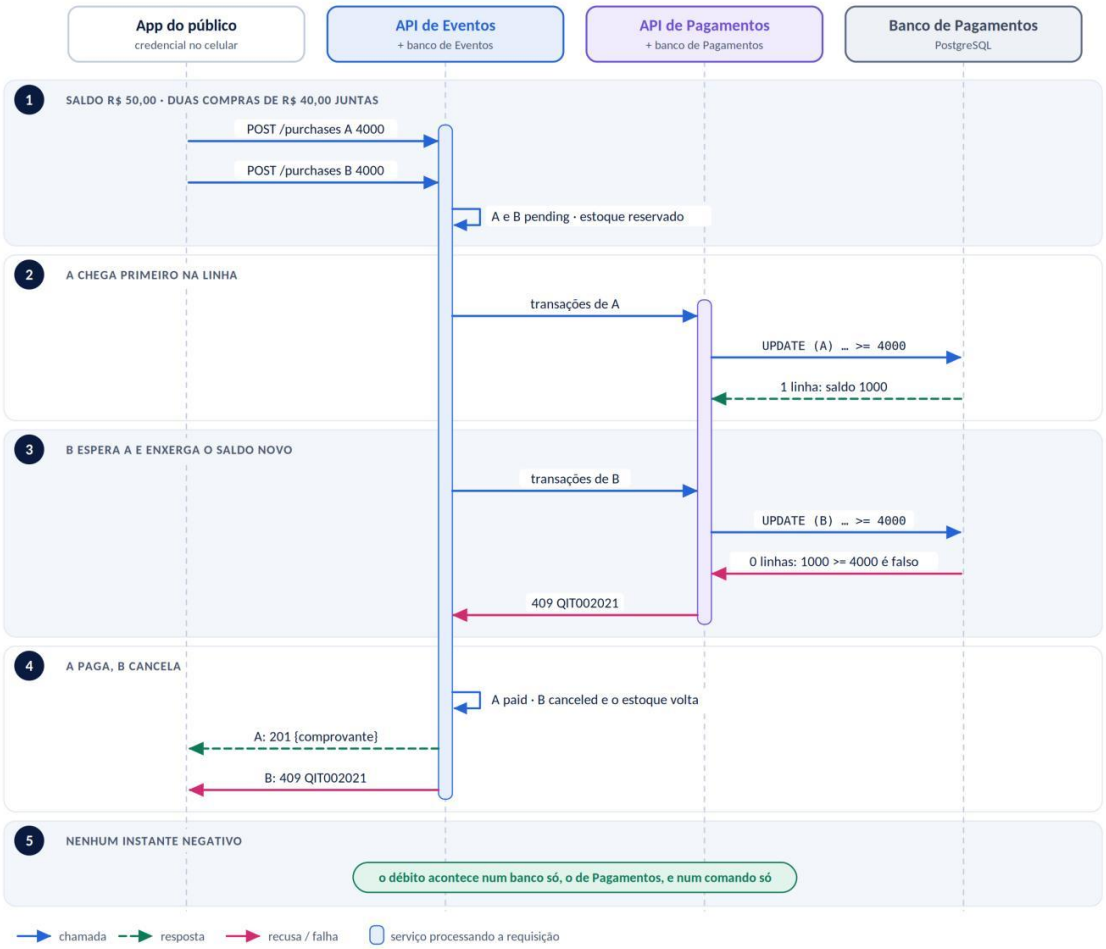


Eventos guarda o que foi comprado. Pagamentos só recebe origem, destino, valor e método.

1. O público escaneia o produto e confirma. O Gateway confere o `INTERNAL-TOKEN` e a sessão e repassa à API de Eventos com o `user_id` de quem chama.
2. A API de Eventos resolve tudo no próprio banco, sem chamar ninguém: formato do corpo, que não tem campo de valor, chave de idempotência, credencial de quem chama e livre, produtos daquele stand e ativos. **O total sai do preço gravado, nunca do app**, a tarifa e o cashback saem das versões vigentes, e o total não pode passar do teto de uma perna (`422 QIT002039`).
3. Numa transação local, baixa o estoque com UPDATE condicional e grava a compra como `pending`, com os itens copiados e as versões das taxas usadas. COMMIT.
4. Pede três transações sob a chave `purchase:{id}`, a venda, a tarifa e o cashback, nessa ordem, sem dizer o que foi comprado. Perna de valor zero não é enviada. Pagamentos executa as três num COMMIT só, com o débito condicional na conta do público. Zero linhas: nada é gravado, e Eventos cancela a compra e devolve o estoque.
5. Com o `201`, ou com o `200` que repete a mesma resposta, Eventos marca a compra como `paid`, grava a tarifa em `fee_charge` e confirma. O comprovante volta ao app e pode ser consultado de novo em `GET /purchases/{id}`.

Numa compra de R$ 40,00 com tarifa de 5% e cashback de 2%: público → stand R$ 40,00. Stand → `fee` R$ 2,00. `fee` → público R$ 0,80. A soma é zero, e o stand fica com R$ 38,00.

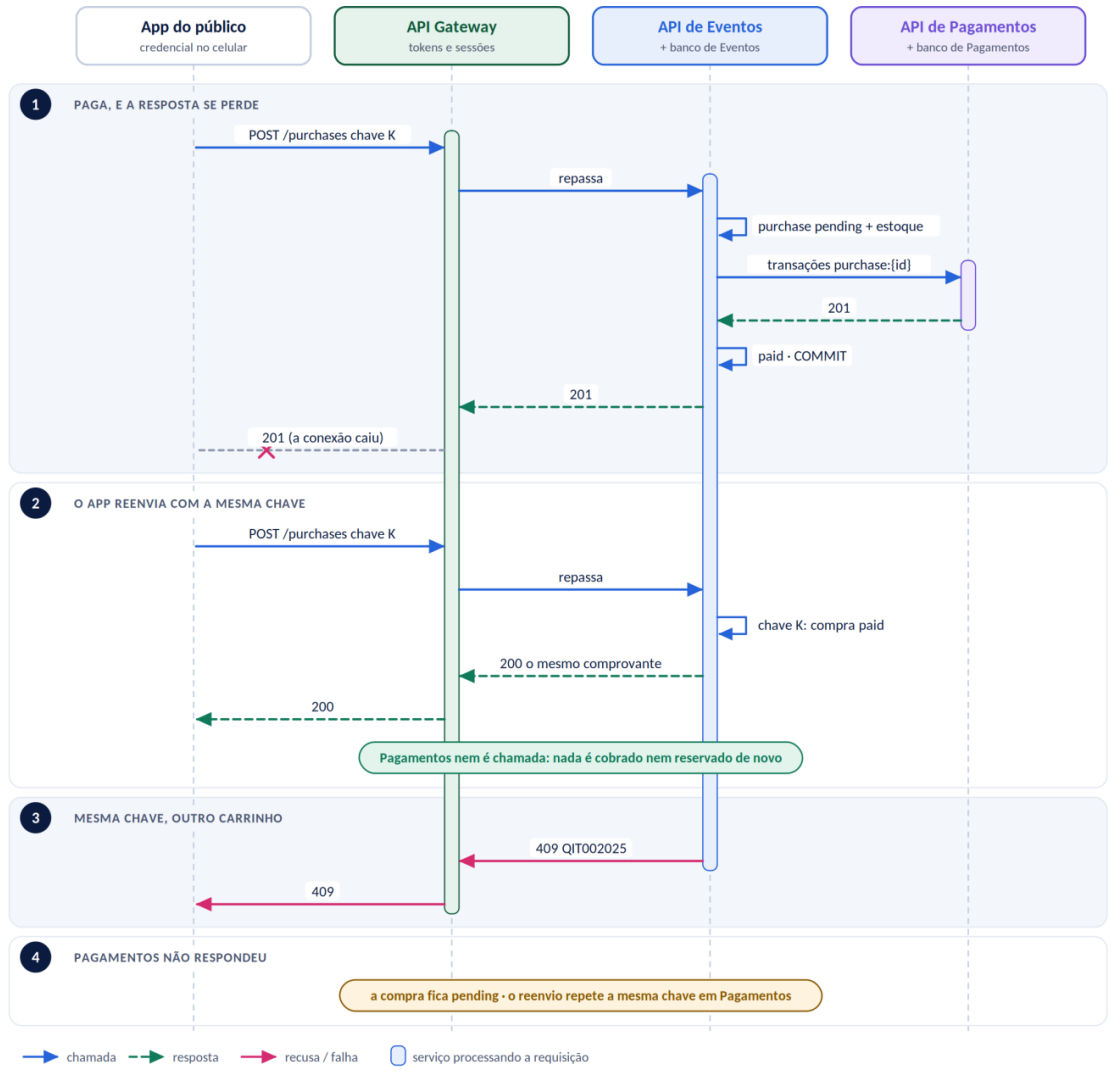
Compra - falha: duas compras disputam o mesmo saldo



A segunda encontra zero linhas, porque a condição é parte da escrita.

1. A credencial tem R$ 50,00 e duas compras de R$ 40,00 chegam juntas. Eventos grava as duas como `pending` e reserva o estoque das duas: ela não tem como saber o saldo, que é de Pagamentos.
2. As transações de A chegam primeiro à linha da conta: uma linha afetada, saldo vai a R$ 10,00.
3. As de B executam o mesmo comando. O Postgres serializa as duas escritas na mesma linha, e B enxerga o saldo já debitado: zero linhas, porque `1000 >= 4000` é falso. Nada de B é gravado.
4. Eventos marca A como `paid` e B como `canceled`, devolvendo o estoque de B no mesmo COMMIT.

Compra - falha: a resposta se perde e o app reenvia



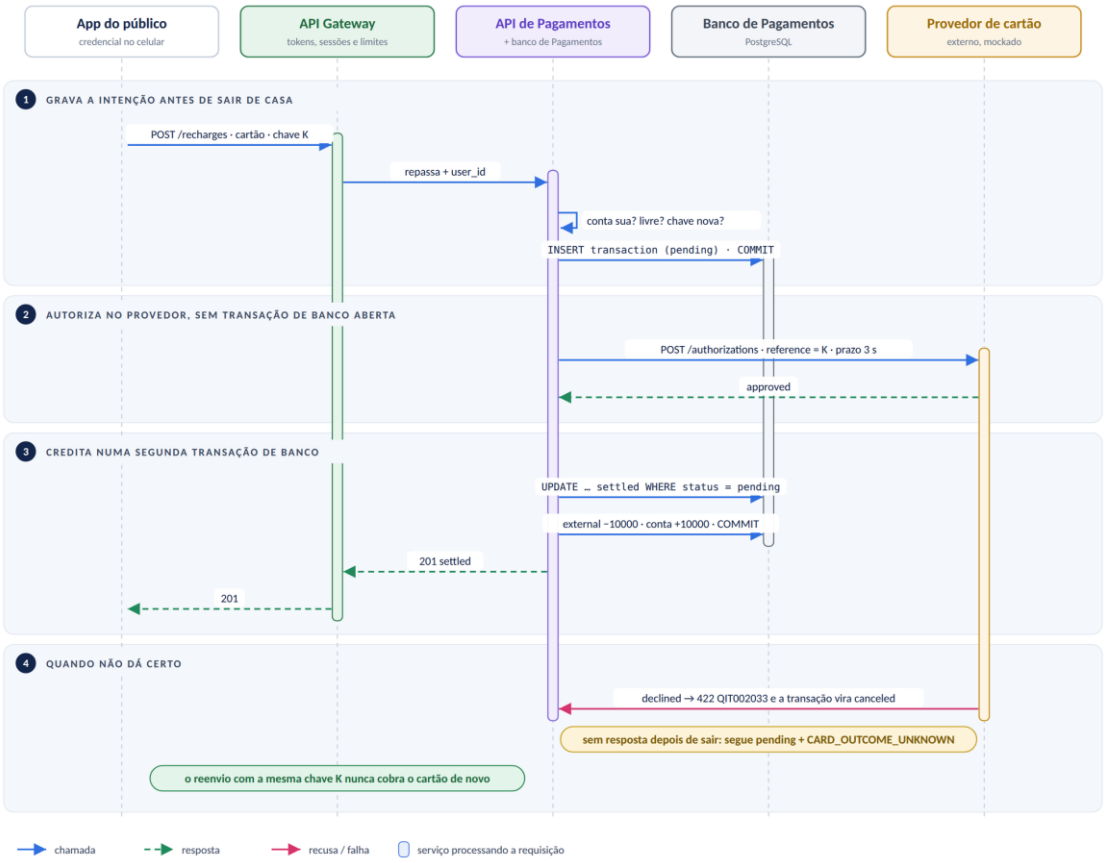
A mesma chave nos dois serviços: perguntar de novo nunca cobra de novo.

1. O app manda a compra com a chave K, gerada uma vez por compra e não uma por envio. A compra vira `paid`, e o `201` se perde porque a conexão caiu.
2. O app reenvia com a mesma chave. Eventos acha a compra `paid` com os mesmos itens e devolve o mesmo comprovante com `200`. Pagamentos nem é chamada. Se a compra foi `canceled` por falta de saldo ou credencial bloqueada, o reenvio repete a mesma recusa, com o mesmo código, guardado em `cancel_code`: um `200` ali faria o app achar que pagou.
3. A mesma chave com outro carrinho é erro de quem chama: `409 QIT002025`. Se as duas requisições chegam juntas, o `UNIQUE` barra o segundo INSERT.
4. Se Pagamentos não respondeu, a compra fica pending com o estoque reservado. O reenvio, ou a rotina de pendentes da API de Eventos, que a cada 30s revisa as pendentes com mais de 30 s, chama de novo com a mesma chave `purchase:{id}`. Se a resposta de Pagamentos for uma recusa (4xx), nada andou, e a compra é cancelada. Persistindo sem resposta por 5 minutos, nasce o log PURCHASE_OUTCOME_UNKNOWN, uma vez por compra.

Recargas - as três entradas de dinheiro

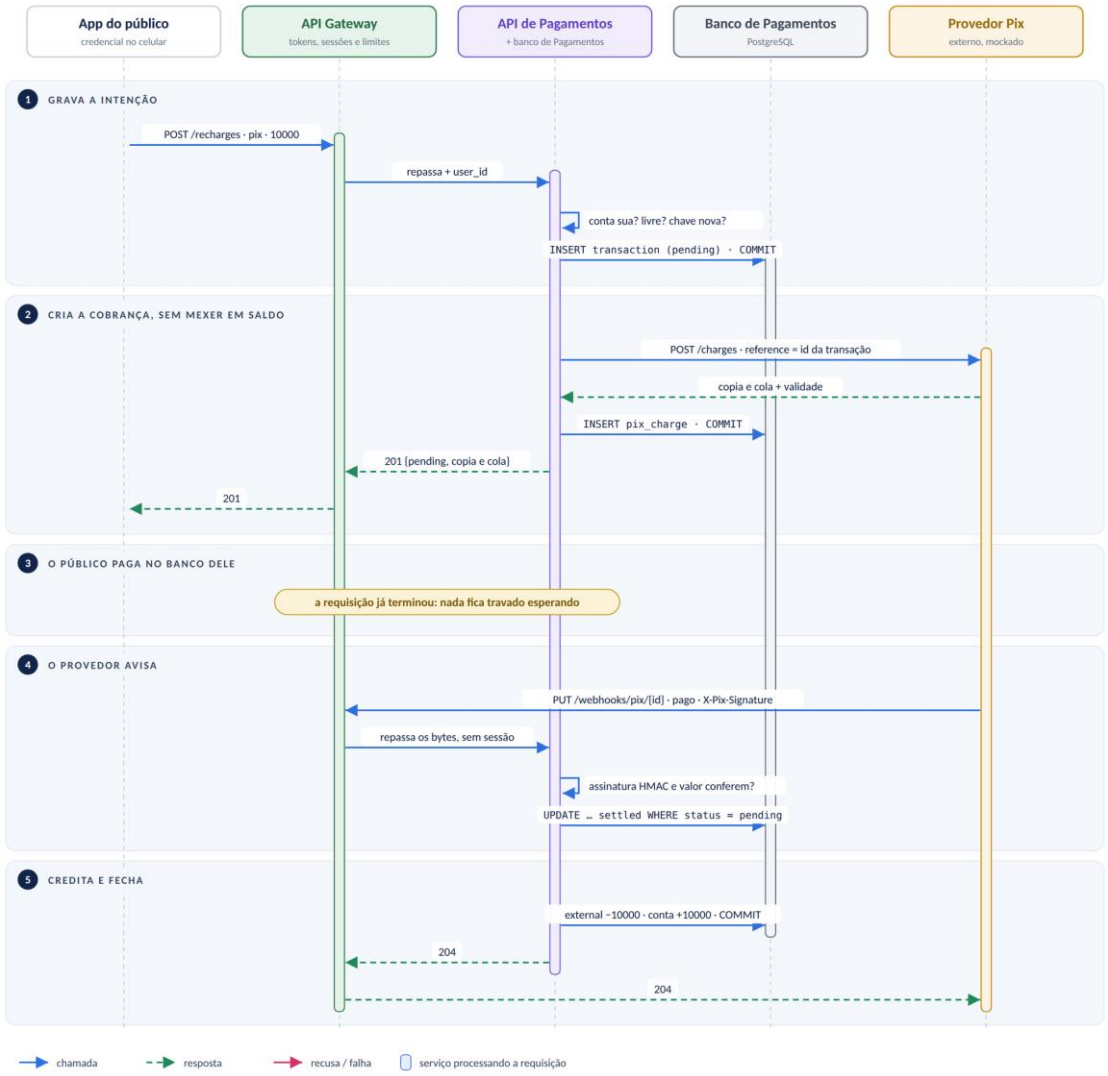
Em **dinheiro**, só o organizador lança, pelo painel, sob a chave `cash:` derivada do evento e da chave do painel: espécie não tem prova externa, e o log guarda quem lançou. Nenhum provedor é chamado, e a credencial não pode estar bloqueada. A conta external do evento fica negativa de propósito, ela é o espelho do que entrou de fora.

Por **cartão**, o app chama Pagamentos pelo Gateway, sem passar por Eventos. Ela decide sozinha: a conta é de quem chama (senão `404 QIT002020`), não está bloqueada (`409 QIT002022`), o evento tem a conta external (`409 QIT002037`) e a chave é nova, tudo antes de qualquer contato com o provedor. O número do cartão nunca passa pelas APIs, só o card_token do provedor. Pagamentos grava a transação external → conta como pending, com a chave, e faz COMMIT, um segundo pedido com a mesma chave encontra essa linha e recebe a recarga como ela está, sem chamar o provedor. Só então pede a autorização, com timeout, fora de qualquer transação de banco.



A intenção é gravada antes de atravessar a fronteira, e o dinheiro só anda depois da resposta.

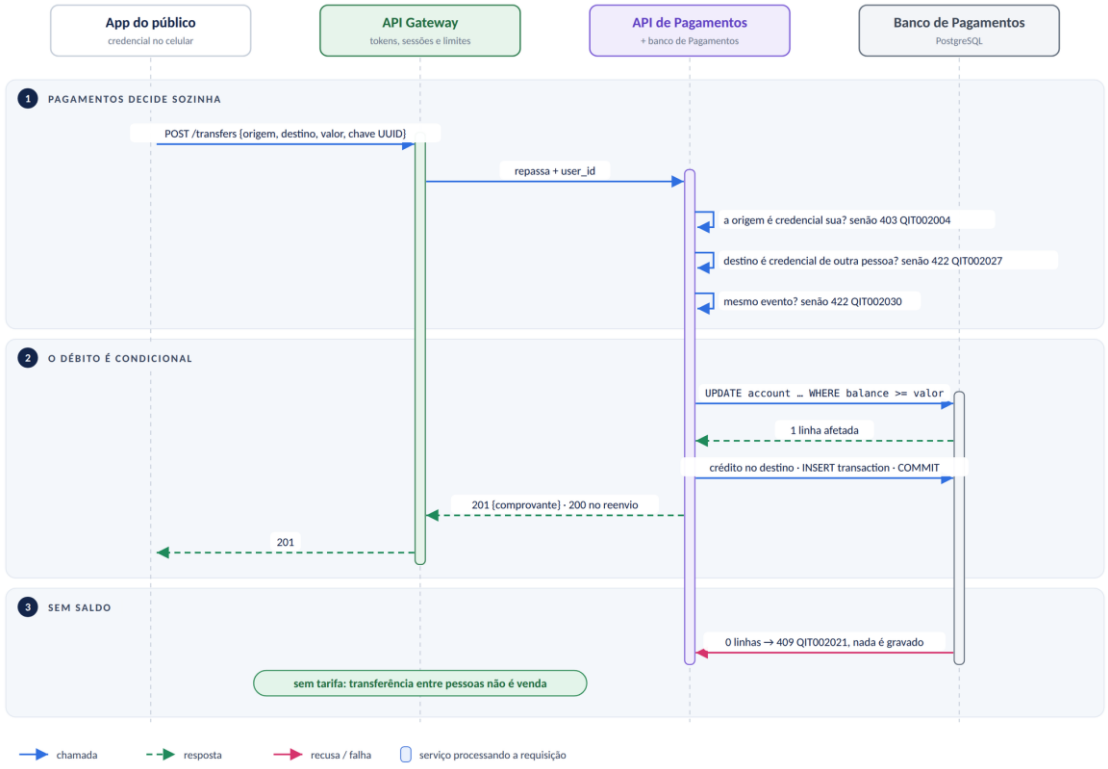
1. Recusado: `422 QIT002033`, e a transação vira canceled. Provedor com 500, sem corpo JSON, ou conexão que não chegou a sair (recusada, nome que não resolve, prazo de conexão): `502 QIT002031`, e a transação também vira canceled. Nos dois casos nenhum saldo mexeu, e o reenvio com a mesma chave devolve a transação canceled com 200.
2. Qualquer falha **depois** de o pedido sair (prazo de resposta, conexão derrubada, resposta cortada) deixa a transação pending e grava CARD_OUTCOME_UNKNOWN, a pessoa pode ter sido cobrada. Quem recebe o alarme consulta o provedor pela reference e conclui à mão. Não existe rota para isso no protótipo.
3. Aprovado, uma segunda transação de banco relê a transação, passa para settled e move o saldo, mesmo que a conta tenha sido bloqueada no meio do caminho, a pessoa já foi cobrada. Se essa gravação falhar, a linha pending fica e o log é CARD_AUTHORIZED_NOT_CREDITED.



Pagamento e crédito em duas requisições: o saldo só entra quando o provedor confirma.

1. No Pix, Pagamentos confere a conta, o bloqueio, a conta external e a chave, grava a transação external → conta como pending e faz COMMIT. Só então pede a cobrança ao provedor, mandando o id da transação como reference, e grava o copia e cola em pix_charge, sem mexer em saldo. O reenvio devolve o mesmo copia e cola. Provedor fora: `502 QIT002031`, e a transação vira canceled. Uma falha depois do envio pode deixar uma cobrança órfã no provedor, que expira sozinha porque a pessoa nunca recebeu o cópia e cola.
2. O provedor chama PUT /webhooks/pix/{id}, que passa sem sessão e é autenticado pela assinatura HMAC-SHA256 do corpo inteiro, inclusive a reference. Um aviso assinado só vale para a cobrança dele: o aviso de uma cobrança na URL de outra recebe `401 QIT002034` e grava PIX_INVALID_SIGNATURE. Assinatura ausente ou errada recebe o mesmo 401, mas vai só para o log comum. Valor diferente do cobrado é `422 QIT002035` com PIX_AMOUNT_MISMATCH.
3. O status muda de pending para settled com UPDATE condicional, e o saldo entra no mesmo COMMIT, mesmo que a conta tenha sido bloqueada depois do pedido: o dinheiro já saiu do banco da pessoa. Um segundo aviso encontra zero linhas e responde 204 sem creditar de novo. Pix expirado vai para canceled.

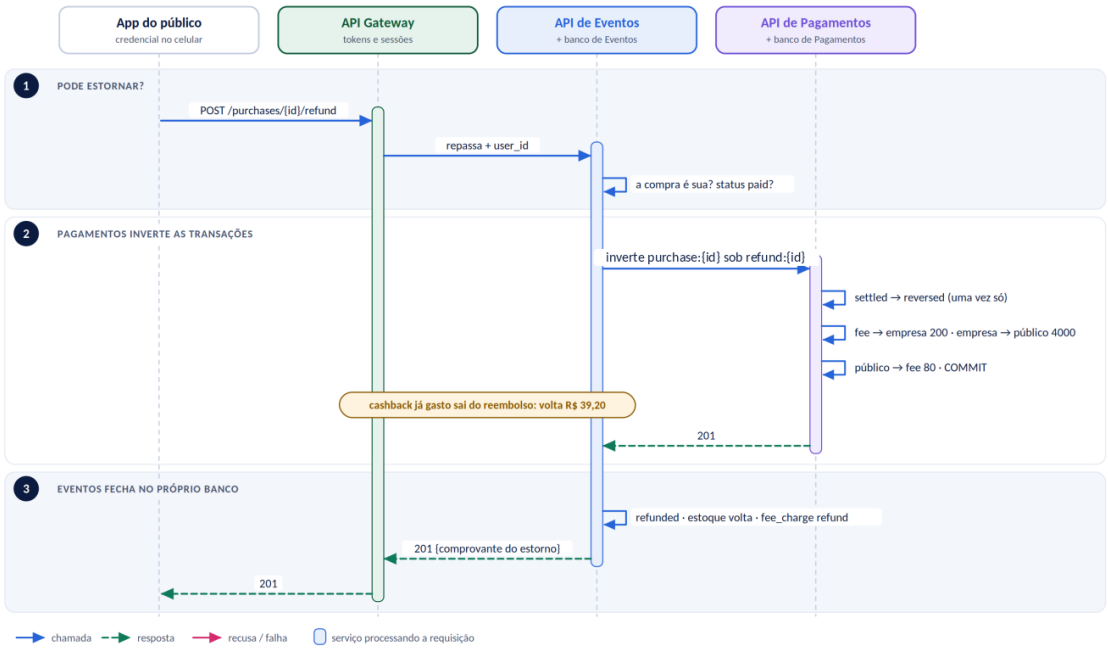
Transferência entre credenciais



Só entre duas pessoas: pagar o stand é uma compra, que cobra a tarifa e baixa o estoque.

1. O app chama Pagamentos direto, com uma chave UUID gerada por ele. Pagamentos confere sozinha: a origem é de quem chama (`404 QIT002020`) e é uma credencial (`403 QIT002004`), as duas contas são do mesmo evento (`422 QIT002030`) e não são a mesma (`422 QIT002029`).
2. O destino precisa ser a credencial de outra pessoa. Stand, fee, external e credencial bloqueada recebem a mesma recusa, `422 QIT002027`, para não revelar a quem conhece o id que aquela credencial está bloqueada. Origem bloqueada é `409 QIT002022`.
3. O débito é o mesmo UPDATE condicional da compra, e não há tarifa: não houve venda.

Estorno



Desfazer é um fato novo: a compra continua no extrato, ao lado da devolução.

1. A pessoa pede pelo app. A empresa e o organizador também podem, pelo painel. Eventos confere quem é quem e se a compra está `paid`.

2. Pede a Pagamentos que inverta as transações da chave `purchase:{id}`, sob a chave `refund:{id}`.
3. Pagamentos marca as originais como `reversed` com UPDATE condicional, o que impede estornar duas vezes, e grava uma transação contrária para cada uma, apontando para a original. **Primeiro saem as que partem de contas do sistema**, então o reembolso chega antes, um cashback já gasto sai do próprio reembolso, e o saldo da pessoa sobe R$ 39,20, não R$ 40,00. Empresa sem saldo para devolver nada muda, `409 QIT002021`.
4. Eventos marca a compra como refunded, guarda o motivo, quem pediu e quando (um CHECK exige os três), devolve o estoque e grava a devolução da tarifa em fee_charge, com valor negativo, no mesmo COMMIT. O estorno ignora o bloqueio dos dois lados, credita o comprador bloqueado e tira do stand bloqueado o que ele deve devolver. Sem resposta de Pagamentos, a compra continua paid, e o pedido seguinte usa a mesma chave `refund:{id}`.

Extrato e fechamento

1. O extrato é `GET /accounts/{id}/transactions`, pedido direto a Pagamentos, que confere que a conta é de quem chama. Os outros recebem 404. Ele é paginado por limit (1 a 100) e page (a partir de 1), com filtro opcional de data.
2. Cada linha é uma transação em que a conta aparece como origem ou destino, já settled ou reversed, da mais nova para a mais antiga, com `amount_cents` com sinal (positivo entrou, negativo saiu), o `balance_after_cents` e o `settled_at`. O saldo corrido sai da própria consulta (`SUM(...) OVER (ORDER BY settled_at, id)`). settled_at é o momento em que o dinheiro andou. Um gatilho o preenche, e ele não muda mais. Por isso um Pix pedido antes de uma transferência e pago depois dela entra no extrato depois dela, e o saldo das linhas passadas nunca muda. date_from e date_to filtram pelo settled_at: sem fuso, valem em America/Sao_Paulo, e date_to só com data inclui o dia inteiro.
3. O fechamento é pedido a Eventos, que confere que quem chama organiza o evento e pergunta a conciliação a Pagamentos.
4. Pagamentos confere duas coisas e devolve `sum_cents`, `accounts` e `mismatches`: a soma dos saldos do evento é zero, com a external negativa espelhando o que entrou de fora. E o saldo guardado de cada conta é igual ao que o histórico dá. Eventos junta a isso o resumo das compras por status e a tarifa líquida. Soma diferente de zero ou qualquer divergência grava RECONCILIATION_MISMATCH.

Perguntas em aberto

O que o time ainda precisa decidir, com a posição que esta RFC defende e o que custa a outra.

5. **Como a pessoa saca a sobra de saldo, e como o evento encerra?** A sobra é da pessoa: vai para o próximo evento dela, ou ela saca. Falta decidir quem aprova o saque, por qual meio o dinheiro sai e em que prazo. A rota que encerra o evento depende dessa resposta. Ao encerrar, Eventos fecha as vendas, mas recarga por cartão ou Pix e transferência são de Pagamentos, que não conhece o estado do evento. Defendemos que o encerramento bloqueie em lote as contas do evento em Pagamentos, e que só o saque ou a passagem para o próximo evento movam a sobra depois disso.
6. **A conclusão do `CARD_OUTCOME_UNKNOWN` continua manual?** Defendemos uma rota interna que pergunta ao provedor pela reference e conclui a recarga sozinha, assim que houver provedor real. Até lá, a ação humana do alarme P2 basta.
7. **A transação ganha a coluna `type`?** Hoje a chave explica a origem de cada movimento. Defendemos esperar o extrato detalhado, com o nome do produto, entrar no escopo: antes disso, a coluna seria uma informação que ninguém lê.
8. **O `POST /events` deve ser idempotente?** Defendemos que sim: o painel manda uma chave, e o id do evento sai dela (`uuid5` do organizador e da chave). Assim o clique duplo não cria dois eventos, e o reenvio reaproveita as contas fee e external.

Principal desafio

Qual é: garantir que nenhuma conta de usuário fique negativa e que cada cobrança aconteça uma vez só, com várias requisições cobrando a mesma credencial ao mesmo tempo, com a conexão caindo no meio e com a compra e o dinheiro morando em bancos diferentes.

Por que é difícil: as três soluções óbvias têm a mesma falha, uma janela entre decidir e gravar. Conferir o saldo e depois debitar deixa as duas requisições lerem o mesmo número antes de qualquer escrita. Consultar se a chave já existe e depois inserir deixa as duas passarem pela consulta antes do primeiro INSERT. E a divisão em serviços acrescenta uma terceira janela, entre o COMMIT da compra e o COMMIT do dinheiro, que nenhuma transação de banco cobre.

Como o desenho resolve: a condição vira parte da escrita. O débito é `UPDATE account SET balance_cents = balance_cents - :amount WHERE id = :id AND balance_cents >= :amount`: o Postgres serializa as escritas na mesma linha, e zero linhas afetadas quer dizer "não tinha saldo", nunca "saldo sumiu". A idempotência usa a mesma ideia, quem fecha a janela é o `UNIQUE`, não a consulta prévia. E a janela entre os dois serviços é fechada pela ordem fixa (grava `pending`, chama, confirma) com uma chave derivada do que originou a chamada, o que torna perguntar de novo seguro. O `CHECK (kind <> 'user' OR balance_cents >= 0)` fica como última defesa.